

RAPTOR EXPERIMENTS

1. Greatest of 3 numbers

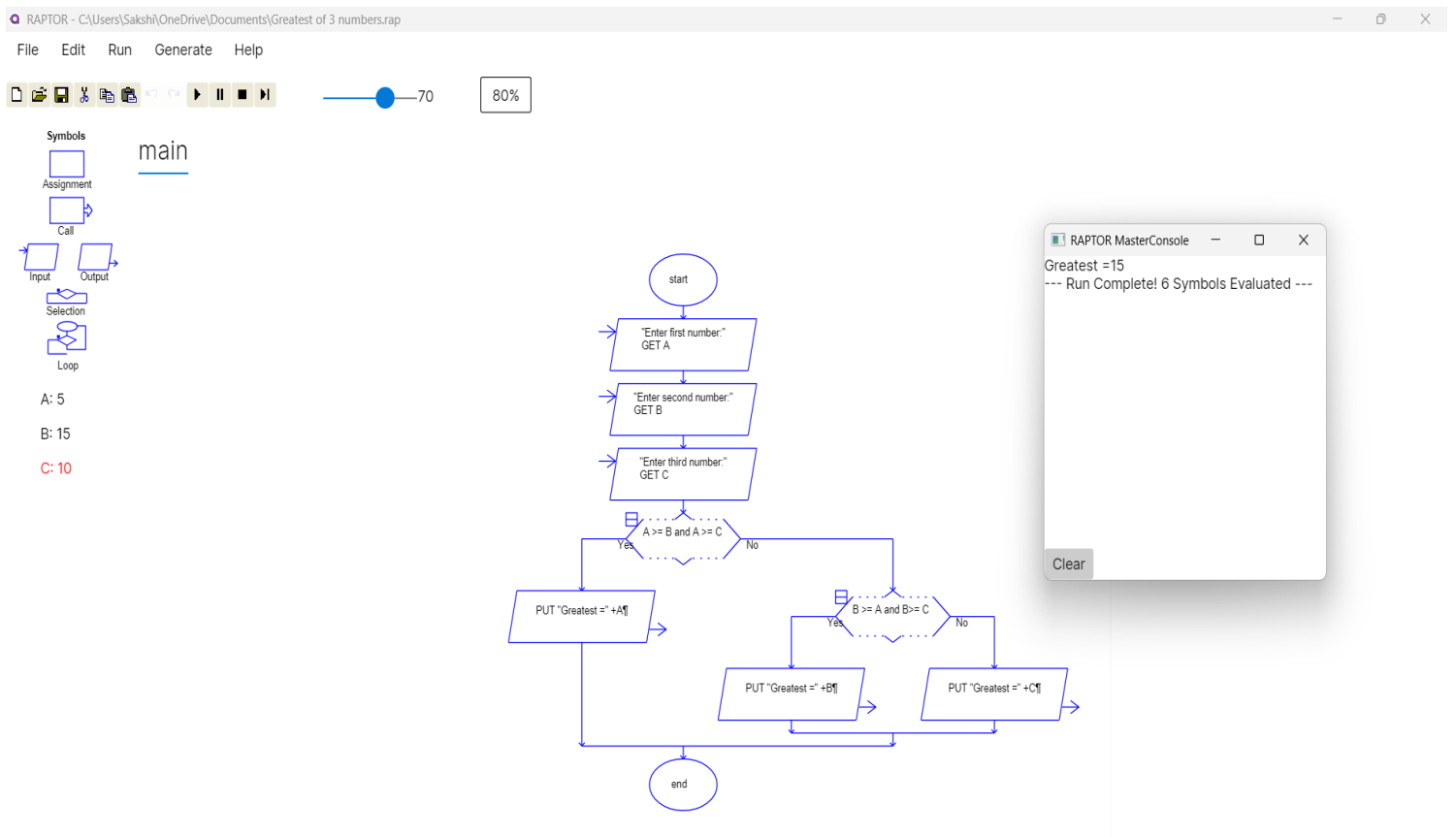
Aim:

To develop a RAPTOR flowchart to determine the greatest among three given numbers.

Algorithm:

1. Read three numbers A, B, and C.
2. Compare A with B and C.
3. If A is the greatest, display A.
4. Otherwise, compare B with C and display the greater number.
5. Display the greatest number obtained after comparison.

Output:



Result:

Thus, the RAPTOR flowchart to find the greatest among three numbers was executed successfully, and the greatest number was displayed correctly.

2. Armstrong Number

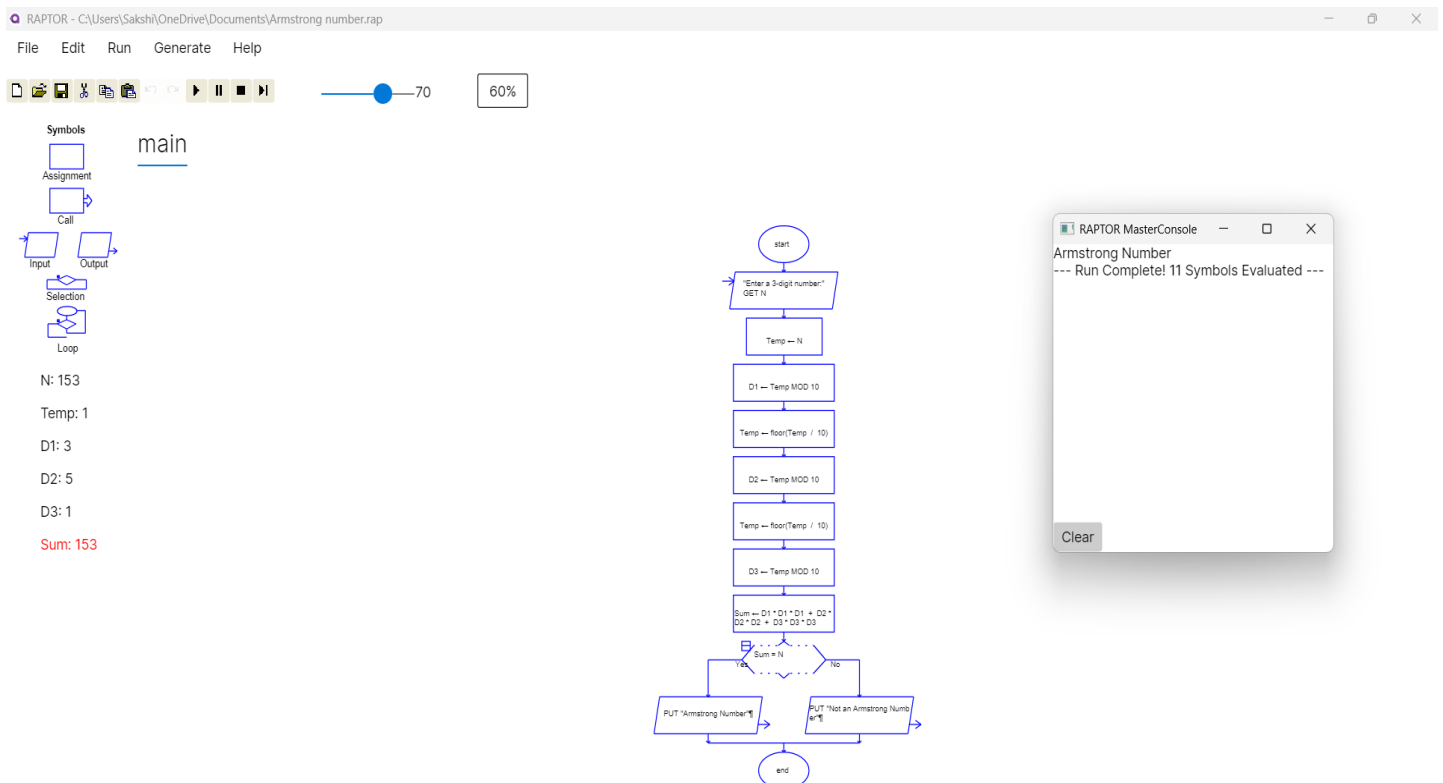
Aim:

To develop a RAPTOR flowchart to check whether a given number is an Armstrong number or not.

Algorithm:

1. Read the given number and store a copy of it.
2. Extract each digit of the number one by one.
3. Find the cube of each extracted digit and calculate their sum.
4. Compare the calculated sum with the original number.
5. If both are equal, display "Armstrong Number"; otherwise, display "Not an Armstrong Number".

Output:



Result:

Thus, the RAPTOR flowchart to check whether a given number is an Armstrong number was executed successfully.

3. Factorial of a number

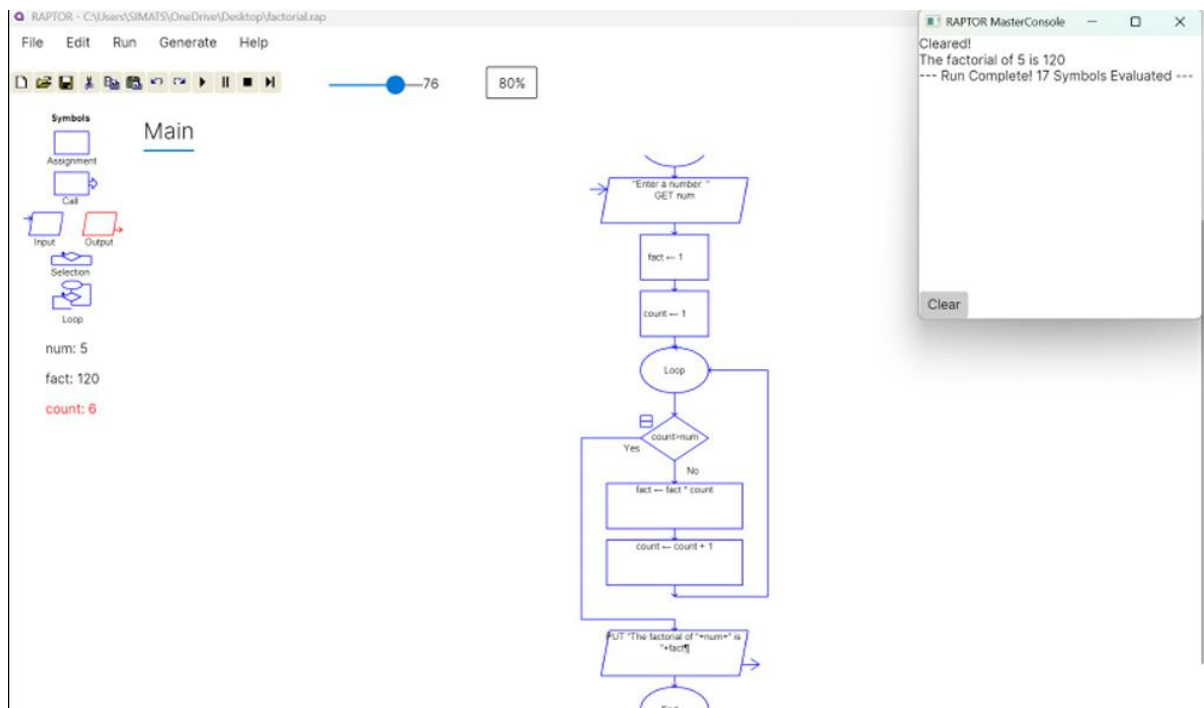
Aim:

To develop a RAPTOR flowchart to find the factorial of a given number.

Algorithm:

1. Read the given number.
2. Initialize the factorial value as 1.
3. Multiply the factorial value by each number from 1 up to the given number.
4. Repeat the multiplication until all numbers are processed.
5. Display the calculated factorial value.

Output:



Result:

Thus, the RAPTOR flowchart to find the factorial of a given number was executed successfully, and the factorial was displayed correctly.

4. To Find the Total Number of Characters in a String

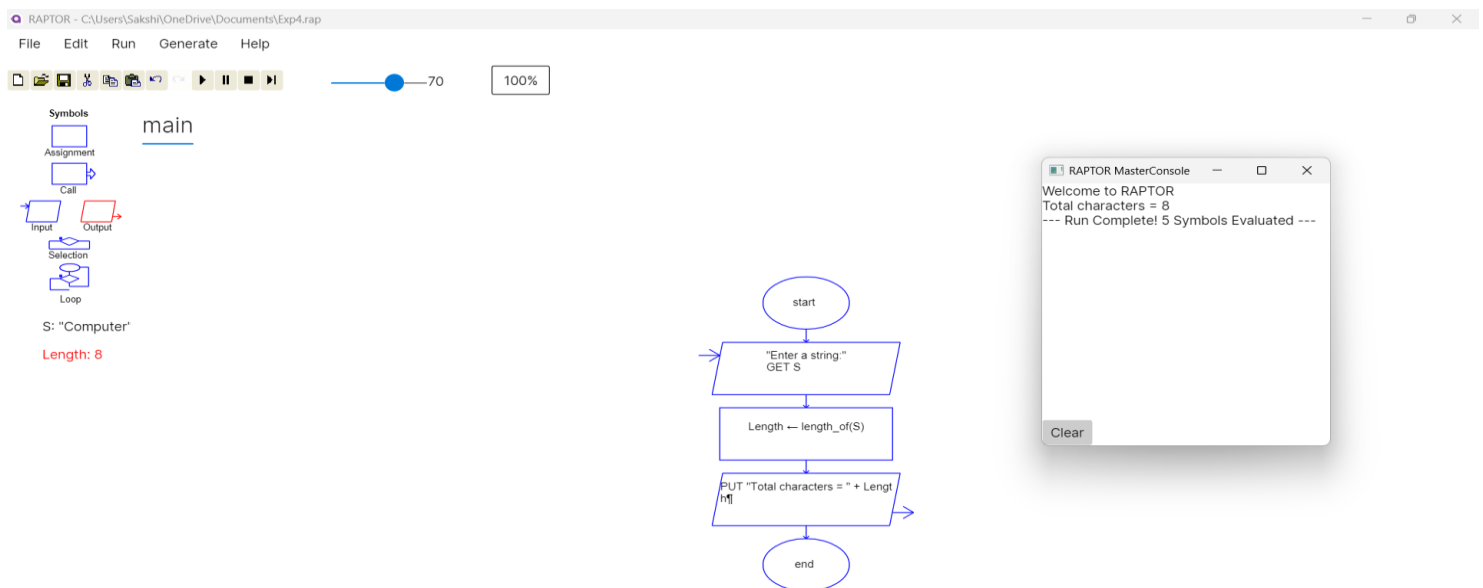
Aim:

To develop a RAPTOR flowchart to find and display the total number of characters in a given string.

Algorithm:

1. Read the input string.
2. Determine the total number of characters in the string.
3. Store the character count in a variable.
4. Display the total number of characters.
5. Stop the execution.

Output:



Result:

Thus, the RAPTOR flowchart to find the total number of characters in a string was executed successfully, and the character count was displayed correctly.

5. Flowchart to Generate the Lexicographically Smallest String by Swapping Two Characters

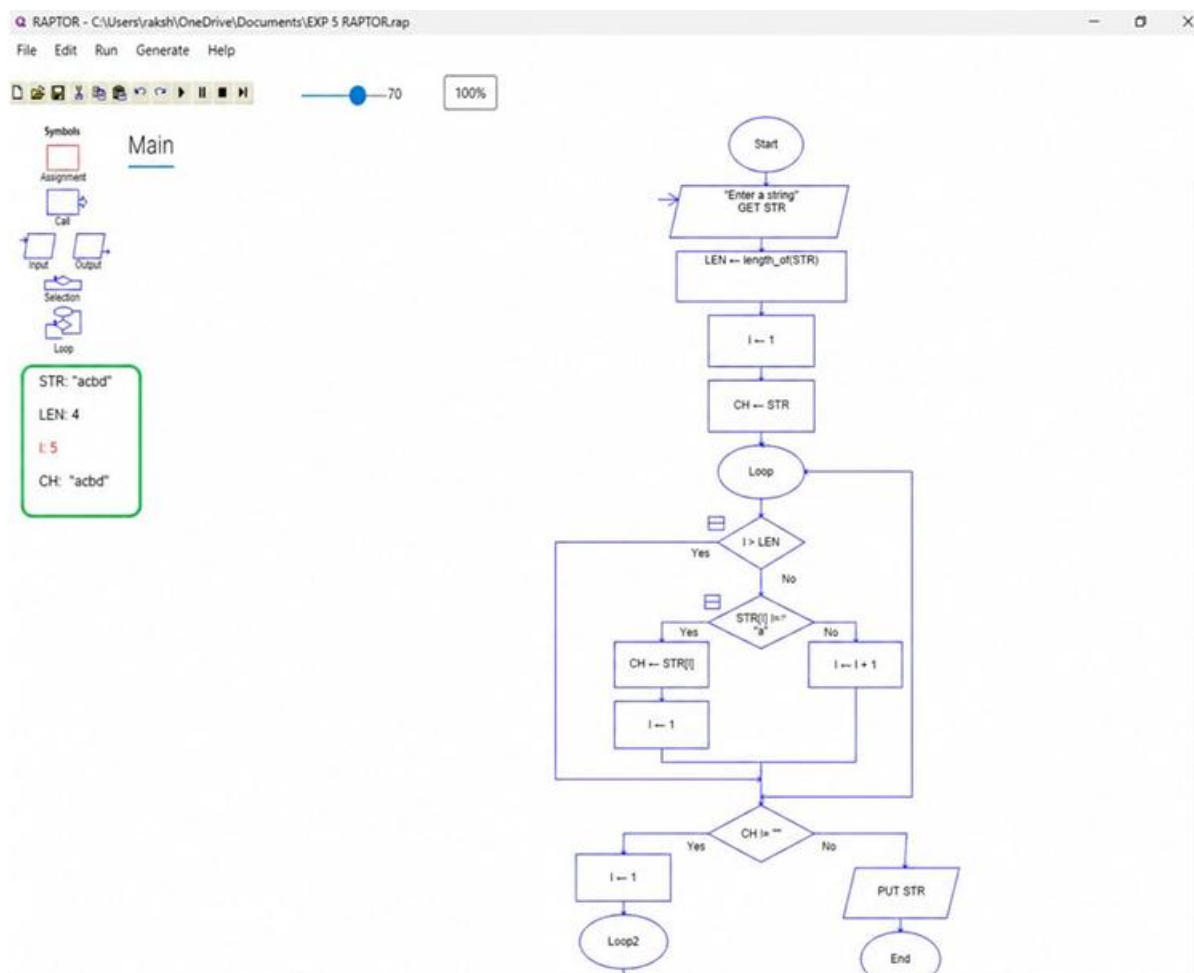
Aim:

To develop a RAPTOR flowchart to replace all occurrences of one character with another character in a given string.

Algorithm

1. Start the program and input the given string.
2. Find the length of the string.
3. Compare the characters from left to right to identify a beneficial swap.
4. Swap the two characters to obtain the lexicographically smallest string.
5. Display the modified string and stop the program.

Output:



RESULT :

Thus the program to execute the lexicographically smallest string was executed successfully .

6. To Calculate Cyclomatic Complexity of a Software Program Using RAPTOR

Aim

To calculate the Cyclomatic Complexity of a flow graph using RAPTOR.

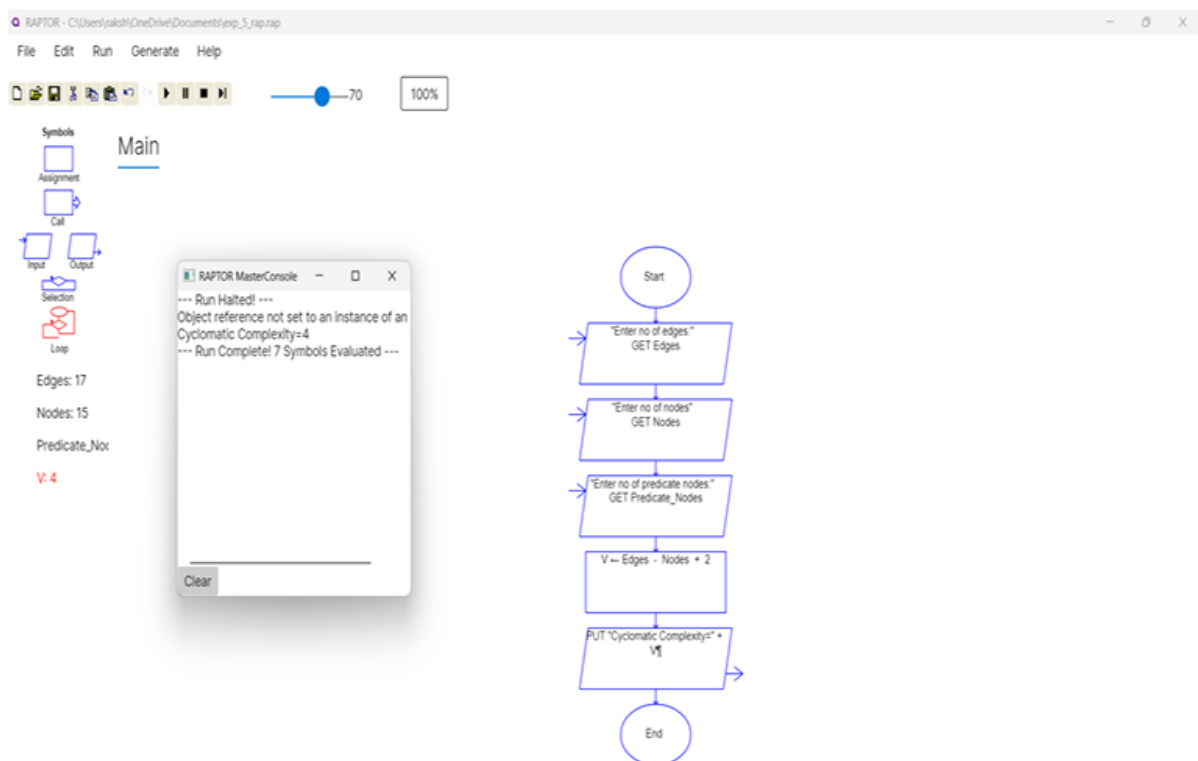
Algorithm

1. Start the program.
2. Read the number of edges (E), nodes (N), and predicate nodes (P).
3. Calculate Cyclomatic Complexity using:

$$CC = E - N + 2P$$

4. Display the Cyclomatic Complexity.
5. Stop the program.

Output:



Result:

Thus, the Cyclomatic Complexity of the given flow graph was calculated successfully using RAPTOR.